

RPi5 GPIO Python 실습

실습 교재

Python GPIO · 디지털 입출력 · PWM · SPI · ADC · I2C · OLED

중앙대학교 전자전기공학부

교재 구성

본 교재는 Raspberry Pi 5에서 Python 언어로 GPIO를 제어하는 방법을 체계적으로 설명합니다.

RPi.GPIO/rpi-igpio 라이브러리 설치부터 시작하여 LED, 버튼, 인터럽트, 피에조 부저, PWM, 서보 모터, 초음파·PIR 센서, SPI 기반 ADC, I2C 기반 기압 센서 및 OLED 디스플레이까지 폭넓은 IoT 하드웨어 제어 실습을 다룹니다.

장	주제	주요 내용
1	Python GPIO 개요 및 환경 설정	rpi-igpio 설치, 핵심 API, BCM 핀 번호, GPIO.cleanup()
2	LED 제어 & 푸시 버튼 & 인터럽트	디지털 출력/입력, 폴업 저항, add_event_detect, bounce_time
3	피에조 부저 & 조도 센서	주파수 생성, softTone 방식, CdS 센서 디지털 입력
4	PWM 제어 (LED · 부저 · 서보 모터)	GPIO.PWM, ChangeDutyCycle, ChangeFrequency, 50Hz 서보
5	초음파 센서 & PIR 센서	HC-SR04 거리 계산, time.time(), HC-SR501 동작 감지
6	SPI 통신 · ADC · LDR · 조이스틱	spidev 라이브러리, MCP3008 10bit ADC, xfer2, 아날로그 입력
7	I2C 통신 · BMP180 · OLED 디스플레이	smbus, i2cdetect, Adafruit SSD1306, PIL 이미지 렌더링

참고 문헌

서영진, 사물인터넷을 위한 리눅스 프로그래밍 with 라즈베리파이, 제이펍, 2022.

정원석, 사물인터넷을 위한 라즈베리파이3 활용, 광문각, 2017.

최주호·김재범·정동진, IoT 사물인터넷과 OpenAI 인공지능을 위한 라즈베리파이5 정석 2판, 앤써북, 2024.

제 1장 Python GPIO 개요 및 환경 설정

1.1 라즈베리파이의 GPIO

Raspberry Pi 5는 40핀 GPIO 헤더를 통해 외부 하드웨어와 연결됩니다. 각 핀은 소프트웨어로 입력 (Input) 또는 출력(Output) 방향을 자유롭게 설정할 수 있으며, 3.3V HIGH 또는 0V(GND) LOW 두 가지 디지털 신호를 처리합니다. 40핀 중 실제 GPIO로 사용 가능한 핀은 26개이고, 나머지는 3.3V·5V 전원 공급 핀과 GND 핀입니다.

GPIO 핀의 번호 체계는 두 가지입니다. BOARD 방식은 커넥터의 물리적 위치(1~40번)를 그대로 사용하고, BCM(Broadcom) 방식은 SoC 칩 내부의 GPIO 번호를 사용합니다. Python RPi.GPIO 라이브러리에서는 프로그램 초기에 `GPIO.setmode(GPIO.BCM)` 또는 `GPIO.setmode(GPIO.BOARD)`로 한 가지를 선택한 뒤 일관되게 사용해야 합니다. 이 교재에서는 회로도와의 일치성이 높은 BCM 방식을 기준으로 설명합니다.

⚠ GPIO 핀의 허용 전압은 3.3V입니다. 5V 신호를 직접 인가하면 핀이 영구 손상됩니다.

1.2 Python GPIO 라이브러리 설치

Raspberry Pi OS Trixie(Debian 13 기반) 이후 환경에서 기존 RPi.GPIO 패키지는 Raspberry Pi 5의 BCM2712 SoC를 지원하지 않습니다. BCM2712는 이전 모델(BCM2835~BCM2711)과 GPIO 아키텍처가 달라 표준 RPi.GPIO가 작동하지 않기 때문입니다.

대신 `rpi-lgpio` 패키지를 설치합니다. 이 패키지는 RPi.GPIO와 완전히 동일한 API를 제공하면서 Raspberry Pi 5를 완벽하게 지원합니다. 기존 코드에서 `import RPi.GPIO as GPIO` 구문을 그대로 사용할 수 있으며, 어떤 코드 수정도 필요 없습니다.

1.2.1 설치 절차

1 단계: Python 3 버전 확인

```
$ python3 --version
```

2 단계: rpi-lgpio 설치

```
$ sudo apt install python3-rpi-lgpio
```

🔗 기존 `sudo apt install python3-rpi.gpio` 명령어는 RPi5에서 동작하지 않습니다. `python3-rpi-lgpio`를 설치하면 `import RPi.GPIO as GPIO`가 정상 작동합니다.

3 단계: GPIO 핀 번호 확인

pinout 명령어로 현재 보드의 핀 맵을 시각적으로 확인할 수 있습니다.

```
$ pinout
```

참고: <https://chatgpt.com/share/689581f1-3a0c-8004-a2bc-42d03d49c320>

1.2.2 설치 방법 비교

방법	설명	RPI5 지원
python3-rpi.gpio	공식 패키지, Bullseye 이후 지원 중단	❌ 불가 (BCM2712 미지원)
python3-rpi-igpio	동일한 API를 제공하는 대체 패키지	✅ 완벽 지원

1.3 RPi.GPIO 핵심 함수

rpi-igpio 패키지를 설치하면 아래의 RPi.GPIO 함수들을 그대로 사용할 수 있습니다. 모든 프로그램은 GPIO 설정으로 시작하고, 종료 전 반드시 GPIO.cleanup()을 호출해야 합니다.

1.3.1 초기화 및 설정 함수

함수	설명
GPIO.setmode(GPIO.BCM)	핀 번호를 BCM(Broadcom) SoC 기준으로 설정. 대부분의 예제에서 권장하는 방식
GPIO.setmode(GPIO.BOARD)	핀 번호를 커넥터 물리 위치(1~40) 기준으로 설정
GPIO.setup(n, GPIO.OUT)	n번 핀을 출력 모드로 설정
GPIO.setup(n, GPIO.IN)	n번 핀을 입력 모드로 설정
GPIO.setup(n, GPIO.IN, pull_up_down=GPIO.PUD_UP)	내부 풀업 저항 활성화하여 입력으로 설정. 버튼 등 스위치에 사용
GPIO.cleanup()	사용한 GPIO 핀을 입력 모드로 되돌리고 메모리 해제. 프로그램 종료 전 필수
GPIO.setwarnings(False)	중복 설정 경고 메시지 억제. 개발 중에 편의상 사용

1.3.2 입출력 함수

함수	설명
GPIO.output(n, GPIO.HIGH)	n번 핀에 HIGH(3.3V) 출력. GPIO.output(n, 1)과 동일
GPIO.output(n, GPIO.LOW)	n번 핀에 LOW(0V) 출력. GPIO.output(n, 0)과 동일
GPIO.input(n)	n번 핀의 현재 디지털 상태를 읽어 1(HIGH) 또는 0(LOW) 반환

1.3.3 인터럽트 함수

함수	설명
GPIO.add_event_detect(channel, edge,	지정 핀(channel)의 엣지 변화를 감지하여

callback, bounce_time)	callback 함수를 자동 호출. bounce_time은 채터링 무시 시간(ms)
GPIO.remove_event_detect(channel)	등록된 인터럽트 해제

🔗 edge 인자 값: *GPIO.FALLING (HIGH→LOW), GPIO.RISING (LOW→HIGH), GPIO.BOTH (양쪽 모두)*

1.4 Python GPIO 프로그램 기본 구조

모든 RPi.GPIO 프로그램은 아래의 공통 패턴을 따릅니다. try-finally 블록으로 프로그램이 어떤 이유로 종료되더라도 GPIO.cleanup()이 반드시 실행되도록 보장합니다.

```
import RPi.GPIO as GPIO
import time

GPIO.setmode(GPIO.BCM)          # BCM 핀 번호 체계 사용
GPIO.setwarnings(False)
GPIO.setup(18, GPIO.OUT)        # GPIO18: 출력으로 설정

try:
    # 메인 로직
    GPIO.output(18, GPIO.HIGH)
    time.sleep(1)
    GPIO.output(18, GPIO.LOW)

except KeyboardInterrupt:       # Ctrl+C 처리
    print('\n프로그램을 종료합니다.')
```

finally:

```
    GPIO.cleanup()              # 반드시 실행됨
```

💡 **팁:** *except KeyboardInterrupt* 블록은 사용자가 **Ctrl+C**로 프로그램을 중단할 때 처리됩니다. **finally** 블록은 정상 종료·예외 발생·**Ctrl+C** 모두에서 실행됩니다.

1.5 주의사항

GPIO 전압 규격: 모든 GPIO 핀의 허용 입력 전압은 3.3V입니다. 5V 신호를 직접 연결하면 핀이 파손됩니다.

GPIO.cleanup() 필수: 프로그램 종료 시 GPIO.cleanup()을 호출하지 않으면 핀 설정이 남아 다음 실행 시 경고가 발생하거나 충돌이 생길 수 있습니다.

핀 모드 지정 우선: GPIO.output() 또는 GPIO.input() 호출 전 반드시 GPIO.setup()으로 핀 방향을 설정해야 합니다.

풀업/풀다운 저항: 외부 스위치를 연결할 때 핀이 플로팅(floating) 상태가 되지 않도록 풀업 또는 풀다운 저항을 사용해야 합니다. pull_up_down=GPIO.PUD_UP 옵션으로 내부 저항을 활성화할 수 있습니다.

제 2장 LED 제어 · 푸시 버튼 · 인터럽트

2.1 LED 제어

LED(Light Emitting Diode)는 전류가 흐를 때 빛을 방출하는 발광 소자로, GPIO 출력 제어의 가장 기본적인 실습 대상입니다. LED를 GPIO 핀에 직접 연결하면 과전류로 파손되므로 반드시 전류 제한 저항을 직렬로 연결해야 합니다.

2.1.1 회로 구성

LED의 긴 다리(양극, Anode)를 220Ω 저항을 통해 GPIO 18번(BCM)에 연결하고, 짧은 다리(음극, Cathode)는 GND 핀에 연결합니다. 220Ω 저항은 GPIO의 3.3V 출력에서 LED 순방향 전압(약 2V)을 뺀 나머지 1.3V를 소모하여 전류를 약 5~6mA 수준으로 제한합니다.

⚠ 저항 없이 LED를 GPIO에 직접 연결하면 순간적으로 수십 mA의 전류가 흘러 LED와 GPIO 핀 모두 손상됩니다.

2.1.2 LED 제어 프로그램 (led.py)

아래 프로그램은 실행 시 BCM 핀 번호를 인자로 받아 해당 핀의 LED를 5번 점멸시킵니다. 커맨드라인 인자를 통해 핀 번호를 전달받으므로 코드 수정 없이 다른 핀에도 재사용할 수 있습니다.

```
import RPi.GPIO as GPIO
import time
import sys

def led_control(gpio):
    GPIO.setup(gpio, GPIO.OUT)

    for _ in range(5):
        GPIO.output(gpio, GPIO.HIGH)    # LED 켜기
        time.sleep(1)                   # 1초 대기
        GPIO.output(gpio, GPIO.LOW)     # LED 끄기
        time.sleep(1)

if __name__ == "__main__":
    if len(sys.argv) < 2:
        print(f"Usage: {sys.argv[0]} GPIO_NO")
        sys.exit(1)

    try:
        gpio_no = int(sys.argv[1])
    except ValueError:
        print("GPIO 번호는 정수로 입력해야 합니다.")
        sys.exit(1)

    GPIO.setmode(GPIO.BCM)
    GPIO.setwarnings(False)

    try:
        led_control(gpio_no)
```

```
finally:
    GPIO.cleanup()
```

```
$ python3 led.py 18
```

`sys.argv`는 실행 시 전달된 커맨드라인 인자 목록입니다. `sys.argv[0]`은 스크립트 파일명, `sys.argv[1]`부터 실제 인자입니다. `int()` 변환 시 숫자가 아닌 문자열이 들어오면 `ValueError` 예외가 발생하므로 `try-except`로 처리합니다.

 *if __name__ == '__main__':* 블록은 이 파일이 직접 실행될 때만 해당 코드를 수행하도록 합니다. 다른 파일에서 *import* 할 때는 실행되지 않습니다.

2.2 푸시 버튼 (Tack Switch)

푸시 버튼은 누리는 동안만 회로가 연결되는 순간 점점 스위치입니다. GPIO 입력 핀에 연결하여 버튼 상태를 소프트웨어로 읽을 수 있습니다. 풀업(Pull-up) 저항과 함께 사용하면 버튼을 누르지 않은 상태에서 HIGH(3.3V), 누른 상태에서 LOW(0V)가 읽힙니다.

`GPIO.setup()`의 `pull_up_down=GPIO.PUD_UP` 옵션을 사용하면 외부 풀업 저항 없이 SoC 내부 풀업 저항을 소프트웨어로 활성화할 수 있습니다. 버튼은 GPIO 24번(BCM), LED는 GPIO 18번(BCM)에 연결합니다.

2.2.1 푸시 버튼 폴링 프로그램 (pushsw.py)

폴링(Polling) 방식은 `while` 루프 안에서 `GPIO.input()`을 반복 호출하여 버튼 상태를 지속적으로 확인합니다. 버튼이 눌린 것이 감지되면(LOW) LED를 1초 켜다가 끕니다.

```
import RPi.GPIO as GPIO
import time

SW = 24 # 버튼 핀 (BCM 24)
LED = 18 # LED 핀 (BCM 18)

def switch_control():
    GPIO.setmode(GPIO.BCM)
    GPIO.setup(SW, GPIO.IN, pull_up_down=GPIO.PUD_UP) # 내부 풀업
    GPIO.setup(LED, GPIO.OUT)

    print("스위치를 눌러 LED를 켜보세요. 종료하려면 Ctrl+C")

    try:
        while True:
            if GPIO.input(SW) == GPIO.LOW: # 버튼 눌림
                GPIO.output(LED, GPIO.HIGH)
                time.sleep(1)
                GPIO.output(LED, GPIO.LOW)
                time.sleep(0.01) # CPU 부하 완화
```

```

except KeyboardInterrupt:
    print("\n프로그램을 종료합니다.")
finally:
    GPIO.cleanup()

if __name__ == "__main__":
    switch_control()

```

```
$ python3 pushsw.py
```

2.3 인터럽트 (Interrupt)

인터럽트 방식에서는 GPIO 핀의 신호 변화가 감지될 때 운영체제가 자동으로 콜백(callback) 함수를 호출합니다. 폴링처럼 CPU가 루프를 돌며 상태를 확인할 필요 없이, 이벤트가 없는 동안 CPU는 다른 작업(메인 루프)을 수행할 수 있습니다.

GPIO.add_event_detect() 함수의 bounce_time 파라미터는 채터링(chattering) 방지 기능을 내장합니다. 예를 들어 bounce_time=200으로 설정하면 첫 번째 이벤트 후 200ms 이내의 추가 신호는 무시합니다. 이를 통해 별도의 소프트웨어 디바운싱 코드 없이도 안정적인 버튼 입력 처리가 가능합니다.

2.3.1 인터럽트 프로그램 (interrupt.py)

아래 프로그램은 GPIO 24번에 Falling edge 인터럽트를 등록합니다. 버튼이 눌릴 때마다(HIGH→LOW 전환) KeyHandler 함수가 자동으로 호출되며, 메인 루프는 방해받지 않고 매초 시간을 출력합니다.

```

import RPi.GPIO as GPIO
from time import sleep

def KeyHandler(n):
    print("Key is pressed [%d]" % n)

PIN = 24
GPIO.setmode(GPIO.BCM)
GPIO.setup(PIN, GPIO.IN)

# Falling edge(HIGH→LOW) 인터럽트 등록
GPIO.add_event_detect(PIN, GPIO.FALLING, callback=KeyHandler)

sec = 0
try:
    while True:
        print("sec : %d" % sec)
        sec += 1
        sleep(1)
except KeyboardInterrupt:
    print("\n프로그램을 종료합니다.")
    GPIO.cleanup()

```

```
$ python3 interrupt.py
```

콜백 함수 `KeyHandler(n)`의 인자 `n`은 인터럽트를 발생시킨 **GPIO** 핀 번호입니다. 같은 콜백을 여러 핀에 등록했을 때 어떤 핀에서 이벤트가 발생했는지 구별하는 데 사용됩니다.

🔗 `GPIO.add_event_detect()`에 `bounce_time=200` 옵션을 추가하면 **200ms** 이내 중복 신호를 자동 무시합니다. 채터링이 심한 버튼에 효과적입니다.

제 3장 피에조 부저 & 온도 센서

3.1 피에조 부저 (소프트웨어 방식)

피에조 부저는 전압을 빠르게 ON/OFF 반복하여 특정 주파수의 진동을 만들어 소리를 내는 출력 장치입니다. WiringPi의 `softTone`과 달리, Python에서는 `GPIO.output()`과 `time.sleep()`을 조합하여 직접 주파수 신호를 만들어야 합니다. 이 방식은 CPU가 직접 타이밍을 제어하므로 소프트웨어 방식(Software Tone)이라고 합니다.

부저의 + 단자는 GPIO 25번(BCM)에, - 단자는 GND에 연결합니다.

3.1.1 buzz() 함수의 원리

특정 주파수(frequency)로 duration초 동안 소리를 내려면, 해당 주파수의 주기(period = 1 / frequency)를 계산하고 반주기마다 HIGH/LOW를 전환합니다. 전환 횟수(cycles)는 총 시간을 주기로 나누어 구합니다.

```
period      = 1.0 / frequency      # 한 주기(초)
half_period = period / 2           # 반주기
cycles      = int(duration / period) # 총 반복 횟수
```

3.1.2 학교종 음악 프로그램 (music.py)

```
import RPi.GPIO as GPIO
import time

SPKR = 25  # GPIO25: 부저 핀

notes = [  # 학교종 음계 주파수 (0 = 쉼표)
    391, 391, 440, 440, 391, 391, 330, 330,
    391, 391, 330, 330, 294, 294, 294, 0,
    391, 391, 440, 440, 391, 391, 330, 330,
    391, 330, 294, 330, 262, 262, 262, 0
]

def buzz(frequency, duration):
    """지정 주파수로 duration초 동안 소리 출력"""
    if frequency == 0:
        time.sleep(duration)  # 쉼표: 그냥 대기
        return

    period      = 1.0 / frequency
    half_period = period / 2
    cycles      = int(duration / period)

    for _ in range(cycles):
        GPIO.output(SPKR, GPIO.HIGH)
        time.sleep(half_period)
        GPIO.output(SPKR, GPIO.LOW)
        time.sleep(half_period)

def play_music():
```

```

GPIO.setmode(GPIO.BCM)
GPIO.setup(SPKR, GPIO.OUT)

try:
    for freq in notes:
        buzz(freq, 0.28)          # 각 음 280ms 연주
    finally:
        GPIO.cleanup()

if __name__ == "__main__":
    play_music()

```

```
$ python3 music.py
```

이 방식의 한계는 `time.sleep()`의 정밀도가 운영체제 스케줄러에 의존한다는 점입니다. 다른 프로세스가 CPU를 점유하면 타이밍이 흔들려 음정이 불안정해집니다. 안정적인 음 출력이 필요할 때는 4장의 PWM 방식을 사용하는 것이 좋습니다.

3.2 조도(빛 감지) 센서

CdS 조도 센서(황화카드뮴 셀, 광의존성 저항 LDR)는 빛의 세기에 따라 저항값이 변하는 소자입니다. 밝을 때 저항이 낮아지고 어두울 때 저항이 높아집니다. GPIO는 아날로그 전압을 직접 읽을 수 없으므로, 센서와 고정 저항을 분압기 회로로 구성하여 특정 밝기 임계값을 기준으로 HIGH/LOW 디지털 신호로 판단합니다.

조도 센서는 GPIO 17번(BCM), LED는 GPIO 18번(BCM)에 연결합니다.

3.2.1 조도 센서 제어 프로그램 (light.py)

```

import RPi.GPIO as GPIO
import time

CDS = 17   # 조도 센서 (BCM 17)
LED = 18   # LED (BCM 18)

def cds_control():
    GPIO.setmode(GPIO.BCM)
    GPIO.setup(CDS, GPIO.IN)
    GPIO.setup(LED, GPIO.OUT)

    print("조도 센서 상태를 감지합니다. 종료하려면 Ctrl+C")

    try:
        while True:
            if GPIO.input(CDS) == GPIO.HIGH: # 빛 감지됨
                GPIO.output(LED, GPIO.HIGH) # LED 켜기
                time.sleep(1)
                GPIO.output(LED, GPIO.LOW)
                time.sleep(0.1)
    except KeyboardInterrupt:

```

```
        print("\n프로그램을 종료합니다.")
    finally:
        GPIO.cleanup()

if __name__ == "__main__":
    cds_control()
```

```
$ python3 light.py
```

프로그램 실행 후 손으로 센서를 가리면 **LED**가 꺼지고, 빛을 비추면 **LED**가 1초간 점멸합니다.
time.sleep(0.1)의 짧은 대기를 포함하여 **CPU**가 루프를 지나치게 빠르게 돌지 않도록 합니다.

제 4장 PWM 제어 (LED · 부저 · 서보 모터)

4.1 RPi.GPIO 에서 PWM

PWM(Pulse Width Modulation, 펄스 폭 변조)은 디지털 신호의 HIGH 상태 비율(듀티 사이클)을 조절하여 아날로그적인 전압 효과를 내는 기법입니다. RPi.GPIO 라이브러리는 소프트웨어 기반 PWM을 지원하며, 거의 모든 GPIO 핀에서 사용할 수 있습니다.

4.1.1 PWM 객체 생성 및 주요 메서드

함수/메서드	설명
GPIO.PWM(channel, frequency)	channel 핀에 frequency(Hz) 주파수의 PWM 객체 생성. 아직 신호 출력은 시작 안 됨
pwm.start(duty_cycle)	PWM 신호 출력 시작. duty_cycle: 0(완전 OFF) ~ 100(완전 ON)
pwm.ChangeDutyCycle(duty)	실행 중 듀티 사이클 변경. 0~100 범위
pwm.ChangeFrequency(freq)	실행 중 주파수 변경(Hz). 부저 음계 변경에 활용
pwm.stop()	PWM 신호 출력 중지. GPIO.cleanup() 전에 호출 권장

4.2 PWM LED 밝기 제어 (pwmled.py)

GPIO.PWM(gpio_pin, 1000)으로 1kHz 주파수의 PWM 객체를 생성하고, 듀티 사이클을 0에서 100% 사이로 반복 변화시켜 LED 밝기를 부드럽게 조절합니다. 듀티 사이클이 높을수록 LED는 밝아집니다.

```
import RPi.GPIO as GPIO
import time
import sys

def led_pwm_control(gpio_pin):
    GPIO.setmode(GPIO.BCM)
    GPIO.setup(gpio_pin, GPIO.OUT)

    pwm = GPIO.PWM(gpio_pin, 1000) # 1kHz 주파수 PWM 생성
    pwm.start(0) # 듀티 사이클 0%로 시작

    try:
        for i in range(10000):
            duty = i % 256 # 0~255 반복
            pwm.ChangeDutyCycle(duty * 100 / 255) # → 0~100%
            time.sleep(0.005)
    finally:
        pwm.ChangeDutyCycle(0)
        pwm.stop()
        GPIO.cleanup()

if __name__ == "__main__":
    if len(sys.argv) < 2:
        print(f"Usage: {sys.argv[0]} GPIO_NO")
        sys.exit(1)
    led_pwm_control(int(sys.argv[1]))
```

```
$ python3 pwmlled.py 18
```

duty * 100 / 255 수식은 0~255 범위의 정수 값을 GPIO.PWM이 요구하는 0~100% 범위로 변환합니다.
i % 256은 0, 1, 2, ... 255, 0, 1, ... 순서로 반복하여 LED 밝기가 주기적으로 증가·초기화되는 효과를 만듭니다.

4.3 PWM 피에조 부저 음악 (pwmmusic.py)

GPIO.PWM의 ChangeFrequency()를 이용하면 실행 중 주파수를 바꿔 다양한 음계를 연주할 수 있습니다. 소리를 낼 때는 듀티 사이클을 50%로 설정하고, 침표(주파수 0)에는 ChangeDutyCycle(0)으로 소리를 끕니다. 부저는 GPIO 12번(BCM)에 연결합니다.

```
import RPi.GPIO as GPIO
import time

SPKR = 12    # BCM 12: PWM 지원 핀

notes = [
    391, 391, 440, 440, 391, 391, 330, 330,
    391, 391, 330, 330, 294, 294, 294, 0,
    391, 391, 440, 440, 391, 391, 330, 330,
    391, 330, 294, 330, 262, 262, 262, 0
]

def play_music():
    GPIO.setmode(GPIO.BCM)
    GPIO.setup(SPKR, GPIO.OUT)

    pwm = GPIO.PWM(SPKR, 440) # 초기 주파수 (임의값)
    pwm.start(0)              # 소리 끈 상태로 시작

    try:
        for freq in notes:
            if freq == 0:
                pwm.ChangeDutyCycle(0)    # 침표: 무음
            else:
                pwm.ChangeFrequency(freq)  # 음계 변경
                pwm.ChangeDutyCycle(50)   # 50% 듀티로 출력
                time.sleep(0.28)          # 280ms 음 길이

        pwm.ChangeDutyCycle(0)
    finally:
        pwm.stop()
        GPIO.cleanup()

if __name__ == "__main__":
    play_music()
```

```
$ python3 pwmmusic.py
```

💡 **팁: 3장의 소프트웨어 방식(buzz 함수)보다 이 PWM 방식이 더 안정적인 음정을 출력합니다.**
ChangeDutyCycle(50)은 정확히 절반 주기 동안 HIGH 신호를 유지하여 깨끗한 구형파를 생성합니다.

4.4 PWM 서보 모터 제어

서보 모터는 PWM 신호의 펄스 폭에 따라 회전 각도가 결정되는 정밀 액추에이터입니다. 50Hz(주기 20ms) PWM 신호를 입력받으며, 듀티 사이클에 따라 0도에서 180도까지 회전합니다.

각도	펄스 폭	듀티 사이클	ChangeDutyCycle 값
0도	0.5ms	2.5%	2.5
90도	1.5ms	7.5%	7.5
180도	2.5ms	12.5%	12.5

⚠ 서보 모터의 VCC는 반드시 외부 5V 전원(별도 어댑터, 2A 이상)에서 공급하십시오. RPi5 GPIO의 5V 핀으로 직접 구동하면 과전류로 보드가 영구 손상됩니다. PWM 신호선만 GPIO 18번(BCM)에 연결하고 GND는 RPi와 공통 접지합니다.

4.4.1 서보 모터 프로그램 (pwm servo.py)

```
import RPi.GPIO as GPIO
import time

SERVO_PIN = 18
GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)
GPIO.setup(SERVO_PIN, GPIO.OUT)

servo = GPIO.PWM(SERVO_PIN, 50) # 50Hz: 서보 표준 주파수
servo.start(0)                  # 듀티 사이클 0으로 시작

try:
    while True:
        servo.ChangeDutyCycle(7.5) # 90도 (1.5ms / 20ms = 7.5%)
        time.sleep(1)
        servo.ChangeDutyCycle(12.5) # 180도 (2.5ms / 20ms = 12.5%)
        time.sleep(1)
        servo.ChangeDutyCycle(2.5) # 0도 (0.5ms / 20ms = 2.5%)
        time.sleep(1)
except KeyboardInterrupt:
    servo.stop()
    GPIO.cleanup()
```

```
$ python3 pwm servo.py
```

듀티 사이클 계산: 펄스 폭(ms) ÷ 주기(20ms) × 100 = 듀티 사이클(%). 예를 들어 90도에 해당하는 1.5ms 펄스는 $1.5 \div 20 \times 100 = 7.5\%$ 입니다.

제 5장 초음파 센서 & PIR 센서

5.1 초음파 센서 (HC-SR04)

HC-SR04는 40kHz 초음파를 발사하고 물체에서 반사된 신호가 돌아오기까지의 시간을 측정하여 거리를 계산하는 센서입니다. 측정 범위는 약 2cm~400cm, 정밀도는 약 3mm입니다. Python에서는 `time.time()` 함수로 마이크로초 단위의 시간 차이를 측정하여 거리를 계산합니다.

5.1.1 동작 순서

1. TRIG 핀에 10μs 동안 HIGH 신호를 출력합니다.
2. 센서 내부에서 40kHz 초음파 8개를 발사합니다.
3. 초음파가 물체에 반사되어 돌아오면 ECHO 핀이 HIGH로 전환됩니다.
4. MCU는 ECHO가 HIGH인 시간(start~stop)을 `time.time()`으로 측정합니다.
5. 거리 = 왕복 시간 × 음속(34300 cm/s) ÷ 2로 계산합니다.

TRIG → GPIO 23번(BCM), ECHO → GPIO 24번(BCM)에 연결합니다.

⚠ HC-SR04의 ECHO 핀은 5V 출력입니다. RPi5 GPIO는 3.3V 허용이므로 1kΩ + 2kΩ 분압 저항으로 전압을 낮춰 연결하십시오.

5.1.2 초음파 거리 측정 프로그램 (HC_SR04.py)

```
import RPi.GPIO as GPIO
import time

GPIO.setmode(GPIO.BCM)
GPIO.setwarnings(False)

TRIG = 23
ECHO = 24
print("Distance measurement in progress")

GPIO.setup(TRIG, GPIO.OUT)
GPIO.setup(ECHO, GPIO.IN)

GPIO.output(TRIG, False)
print("Waiting for sensor to settle")
time.sleep(2)          # 센서 안정화 2초 대기

try:
    while True:
        GPIO.output(TRIG, True)    # 트리거 HIGH
        time.sleep(0.00001)        # 10μs 유지
        GPIO.output(TRIG, False)   # 트리거 LOW

        while GPIO.input(ECHO) == 0:
            start = time.time()    # ECHO 상승 시각
```

```

        while GPIO.input(ECHO) == 1:
            stop = time.time()    # ECHO 하강 시각

            check_time = stop - start
            distance = check_time * 34300 / 2    # 거리(cm)
            print("Distance : %.1f cm" % distance)
            time.sleep(0.4)

except KeyboardInterrupt:
    print("Measurement stopped by User")
    GPIO.cleanup()

```

```
$ python3 HC_SR04.py
```

5.2 PIR 모션 센서 (HC-SR501)

PIR(Passive Infrared) 센서는 사람의 체온에서 방출되는 적외선 변화를 감지하여 움직임을 탐지합니다. HC-SR501은 감지 범위 내에서 움직임을 있으면 OUT 핀에 HIGH를 출력하고, 없으면 LOW를 유지합니다. 전원 인가 후 약 5초의 안정화(워밍업) 시간이 필요합니다.

PIR → GPIO 17번(BCM), 빨간 LED → GPIO 20번(BCM), 노란 LED → GPIO 21번(BCM)에 연결합니다.

5.2.1 PIR 센서 프로그램 (HC_SR501.py)

```

import RPi.GPIO as GPIO
import time

led_R = 20    # 빨간 LED (정지 상태 표시)
led_Y = 21    # 노란 LED (동작 감지 표시)
sensor = 17   # PIR 센서 신호

GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)
GPIO.setup(led_R, GPIO.OUT)
GPIO.setup(led_Y, GPIO.OUT)
GPIO.setup(sensor, GPIO.IN)

print("PIR Ready . . . .")
time.sleep(5)    # PIR 안정화 5초 대기

try:
    while True:
        if GPIO.input(sensor) == 1:    # 동작 감지 (HIGH)
            GPIO.output(led_Y, 1)
            GPIO.output(led_R, 0)
            print("Motion Detected !")
            time.sleep(0.2)
        else:                            # 정지 상태 (LOW)
            GPIO.output(led_R, 1)
            GPIO.output(led_Y, 0)
            print("NO Motion !")
            time.sleep(0.2)

except KeyboardInterrupt:
    print("Stopped by User")

```

```
GPIO.cleanup()
```

```
$ python3 HC_SR501.py
```

🔗 HC-SR501에는 두 개의 조절 나사가 있습니다. 하나는 감도(감지 거리 3~7m), 다른 하나는 신호 유지 시간(5초~300초)을 조절합니다.

제 6장 SPI 통신 · ADC · LDR · 조이스틱

6.1 SPI(Serial Peripheral Interface) 통신

SPI는 마스터 장치와 여러 슬레이브 장치 사이의 동기식 직렬 통신 방식입니다. 클럭(SCLK)을 기준으로 데이터를 동기화하며, 마스터에서 슬레이브로 데이터를 보내는 MOSI, 슬레이브에서 마스터로 데이터를 보내는 MISO, 슬레이브를 선택하는 CS(Chip Select) 등 4개의 신호선을 사용합니다.

신호	방향	역할	다른 명칭	설명
SCLK	M→S	클럭	SCK, CLK	통신 동기화용 클럭, 마스터 생성
MOSI	M→S	데이터	SDO, SI	마스터→슬레이브 데이터
MISO	S→M	데이터	SDI, SO	슬레이브→마스터 데이터
CS/SS	M→S	선택	/CE, /SS	LOW일 때 해당 슬레이브 선택

Raspberry Pi 5는 여러 개의 SPI 버스를 지원합니다. 기본 SPI0 버스는 GPIO 10(MOSI), GPIO 9(MISO), GPIO 11(SCLK), GPIO 8(CE0), GPIO 7(CE1) 핀을 사용합니다.

6.2 SPI 활성화 및 spidev 설치

6.2.1 SPI 인터페이스 활성화

기본값으로 비활성화되어 있는 SPI 인터페이스를 `raspi-config`로 활성화합니다.

```
$ sudo raspi-config
```

`raspi-config` 메뉴에서 3. Interface Options → I2 SPI → Enable 을 선택하여 SPI를 활성화합니다. 설정 후 재부팅합니다.

```
$ sudo reboot
```

재부팅 후 SPI 장치 파일이 생성되었는지 확인합니다.

```
$ ls -al /dev/spidev*
```

`spidev0.0`과 `spidev0.1` 파일이 보이면 SPI가 정상적으로 활성화된 것입니다.

6.2.2 py-spidev 라이브러리 설치

```
$ sudo apt-get install python3-dev
$ git clone https://github.com/doceme/py-spidev.git
```

```
$ cd py-spidev
$ make all
```

🔗 Python 3.x 환경에서 빌드 오류가 발생하면 `py-spidev` 디렉토리의 `Makefile`에서 `PYTHON ?= python`을 `PYTHON ?= python3`으로 수정한 후 다시 `make all`을 실행합니다.

6.3 ADC (Analog to Digital Converter) - MCP3008

Raspberry Pi의 GPIO는 디지털 신호만 처리하므로, 가변저항·LDR·조이스틱 등 아날로그 출력 센서를 직접 읽을 수 없습니다. MCP3008은 SPI 통신을 통해 8채널의 아날로그 신호를 10비트 디지털 값(0~1023)으로 변환해 주는 ADC 칩입니다.

10비트 해상도란 입력 전압(0~3.3V)을 $2^{10} = 1024$ 단계로 나누어 표현하는 것입니다. 최소 분해능은 $3.3V \div 1024 \approx 3.22mV$ 이며, 이 값이 디지털 값 1에 해당하는 전압 변화량입니다.

6.3.1 readadc() 함수 원리

MCP3008과 SPI 통신은 3바이트 데이터를 주고받는 방식으로 이루어집니다. `xfer2()` 함수로 `[1, 8 + adcnum << 4, 0]`을 전송하면 MCP3008이 지정 채널의 10비트 ADC 값을 응답합니다.

```
def readadc(adcnum):
    if adcnum > 7 or adcnum < 0:
        return -1                # 유효하지 않은 채널
    r = spi.xfer2([1, 8 + adcnum << 4, 0]) # 3바이트 SPI 전송
    data = ((r[1] & 3) << 8) + r[2]        # 10비트 데이터 조합
    return data                    # 0~1023 반환
```

응답 바이트 `r[1]`의 하위 2비트와 `r[2]` 전체 8비트를 합쳐 10비트 데이터를 구성합니다. `((r[1] & 3) << 8)`은 상위 2비트를 8자리 올리고, `+ r[2]`로 하위 8비트를 더합니다.

6.3.2 가변저항 ADC 읽기 프로그램 (MCP_3008.py)

```
import spidev
import time

delay      = 0.5      # 측정 간격(초)
pot_channel = 0        # MCP3008 채널 0번

spi = spidev.SpiDev()
spi.open(0, 0)         # SPI 버스 0, CE0
spi.max_speed_hz = 100000 # 100kHz

def readadc(adcnum):
    if adcnum > 7 or adcnum < 0:
        return -1
    r = spi.xfer2([1, 8 + adcnum << 4, 0])
    data = ((r[1] & 3) << 8) + r[2]
    return data
```

```
while True:
    pot_value = readadc(pot_channel)
    print("-----")
    print("POT Value: %d" % pot_value)
    time.sleep(delay)
```

```
$ python3 MCP_3008.py
```

6.4 LDR(Light Dependent Resistor) 센서

LDR은 빛의 세기에 따라 저항값이 변하는 광의존성 저항입니다. 주위가 밝으면 저항이 줄어들어 출력 전압이 3.3V에 가까워지고, 어두우면 저항이 커져 출력 전압이 낮아집니다. MCP3008의 ADC를 통해 읽으면 아날로그 밝기 값(0~1023)을 정확하게 측정할 수 있습니다.

LDR 센서 모듈의 출력을 MCP3008 채널 0번에 연결합니다.

6.4.1 LDR 아날로그 값 읽기 프로그램 (LDR.py)

```
import spidev
import time

ldr_channel = 0

spi = spidev.SpiDev()
spi.open(0, 0)
spi.max_speed_hz = 1000000

def readadc(adcnum):
    if adcnum > 7 or adcnum < 0:
        return -1
    r = spi.xfer2([1, 8 + adcnum << 4, 0])
    data = ((r[1] & 3) << 8) + r[2]
    return data

while True:
    ldr_value = readadc(ldr_channel)
    print("-----")
    print("LDR Value: %d" % ldr_value) # 0(어두움) ~ 1023(밝음)
    time.sleep(0.5)
```

```
$ python3 LDR.py
```

6.5 조이스틱 (Joystick)

아날로그 조이스틱은 X축, Y축의 위치를 각각 가변저항으로 출력하고, 조이스틱을 누르면 스위치 신호를 출력합니다. 세 개의 아날로그 채널(VRx, VRY, SW)을 MCP3008의 채널 1, 2, 0에 각각 연결하여 읽습니다.

6.5.1 조이스틱 프로그램 (Joystick.py)

```
import spidev
import time

sw_channel = 0    # 스위치 (눌림: 낮은 값)
vrx_channel = 1   # X축 위치
vry_channel = 2   # Y축 위치

spi = spidev.SpiDev()
spi.open(0, 0)
spi.max_speed_hz = 1000000

def readadc(adcnum):
    if adcnum > 7 or adcnum < 0:
        return -1
    r = spi.xfer2([1, 8 + adcnum << 4, 0])
    data = ((r[1] & 3) << 8) + r[2]
    return data

while True:
    vrx_pos = readadc(vrx_channel)    # X축: 0(왼쪽)~1023(오른쪽)
    vry_pos = readadc(vry_channel)    # Y축: 0(위)~1023(아래)
    sw_val = readadc(sw_channel)      # 스위치
    print("X : {} Y : {} SW : {}".format(vrx_pos, vry_pos, sw_val))
    time.sleep(0.5)
```

```
$ python3 Joystick.py
```

조이스틱의 중앙 위치에서 VRx, VRy 값은 약 512(중간)입니다. 조이스틱을 최대한 움직이면 0 또는 1023에 가까운 값이 출력됩니다. SW 채널은 조이스틱을 누르지 않은 상태에서 높은 값(약 1023), 누른 상태에서 낮은 값(약 0)을 출력합니다.

제 7장 I2C 통신 · BMP180 · OLED 디스플레이

7.1 I2C(Inter-Integrated Circuit) 통신

I2C는 클럭(SCL)과 데이터(SDA) 단 2개의 선으로 마스터 1개와 여러 슬레이브를 연결하는 통신 방식입니다. 각 슬레이브 장치는 고유한 7비트 주소를 가지며, 마스터가 해당 주소를 지정하여 특정 장치와 선택적으로 통신합니다. SPI보다 배선이 단순하고 다수의 장치를 쉽게 연결할 수 있다는 장점이 있습니다.

신호선	핀 번호(BCM)	역할
SDA	GPIO 2 (물리 3번)	데이터선: 마스터-슬레이브 간 양방향 데이터 전송
SCL	GPIO 3 (물리 5번)	클럭선: 마스터가 생성하는 동기화 클럭

🔗 Raspberry Pi 5에는 I2C용 풀업 저항이 이미 내장되어 있으므로 별도 외부 풀업 저항이 필요 없습니다.

7.1.1 SPI vs UART vs I2C 비교

구분	SPI	UART	I2C
신호선	/CE, CLK, MOSI, MISO (4개)	TX, RX (2개)	SCL, SDA (2개)
방식	동기식	비동기식	동기식
속도	빠름 (MHz 이상)	느림 (115200bps)	100kbps / 400kbps / 3.4Mbps
1:N	가능 (CS 핀 추가)	1:1만 가능	가능 (주소 지정)
전송	풀 듀플렉스	풀 듀플렉스	반이중
장점	고속, 다장치	단순, 범용	배선 단순, 주소 기반

7.2 I2C 활성화 및 라이브러리 설치

7.2.1 I2C 인터페이스 활성화

```
$ sudo raspi-config
```

3. Interface Options → I5 I2C → Enable 선택 후 재부팅합니다.

```
$ sudo reboot
```

재부팅 후 I2C 장치 파일을 확인합니다.

```
$ ls -al /dev/i2c*
```

i2c-1 파일이 있으면 I2C가 정상적으로 활성화된 것입니다.

7.2.2 관련 라이브러리 설치

```
$ sudo apt-get install python3-dev
$ sudo apt-get install python3-smbus libi2c-dev
$ sudo reboot
```

7.2.3 I2C 장치 주소 확인

i2cdetect 명령어로 현재 I2C 버스에 연결된 장치의 주소를 확인할 수 있습니다.

```
$ i2cdetect -y 1
```

출력 표에서 숫자(예: 77, 3c)가 있는 위치가 연결된 장치의 16진수 주소입니다. BMP180은 0x77, SSD1306 OLED는 0x3C 주소를 사용합니다.

7.3 BMP180 기압·온도 센서

BMP180은 Bosch에서 제작한 고정밀 기압·온도 센서입니다. I2C 인터페이스(주소 0x77)를 통해 현재 대기압(Pa), 온도(°C), 고도(m)를 측정할 수 있습니다. 고도는 기압과 표준 대기 모델을 이용하여 계산됩니다.

7.3.1 Adafruit BMP180 라이브러리 설치

Adafruit BMP180 전용 Python 라이브러리를 설치하면 센서 초기화와 보정 데이터를 자동으로 처리해 줍니다.

```
$ pip3 install adafruit-circuitpython-bmp180 --break-system-packages
```

🔗 pip 설치 시 `--break-system-packages` 옵션이 필요할 수 있습니다. 또는 가상 환경(venv)을 사용하는 것을 권장합니다.

7.3.2 BMP180 측정 프로그램 (BMP_180.py)

```
import bmpsensor
import time

# 온도, 압력, 고도 값을 한 번에 읽어 변수에 저장
temp, pressure, altitude = bmpsensor.readBmp180()

while True:
    print('Temp      = {0:0.2f} *C'.format(temp))
    print('Pressure = {0:0.2f} Pa'.format(pressure))
    print('Altitude = {0:0.2f} m'.format(altitude))
    print()
    time.sleep(2)
```

```
$ python3 BMP_180.py
```

7.4 OLED 디스플레이 (SSD1306)

SSD1306은 128×64 픽셀 OLED 디스플레이 모듈의 드라이버 IC입니다. I2C(주소 0x3C) 인터페이스를 통해 픽셀 단위로 화면을 제어할 수 있습니다. Adafruit의 SSD1306 라이브러리와 PIL(Python Imaging Library)을 함께 사용하면 텍스트, 도형, 이미지를 자유롭게 표시할 수 있습니다.

7.4.1 Adafruit OLED 라이브러리 설치

```
$ pip3 install adafruit-circuitpython-ssd1306 pillow --break-system-packages
```

7.4.2 OLED + BMP180 통합 프로그램 (Oled.py)

BMP180에서 읽은 온도·기압·고도 값을 OLED 화면에 2초마다 갱신하여 표시합니다.

```
import time
import board
from PIL import Image, ImageDraw, ImageFont
import adafruit_ssd1306
import bmpsensor

# I2C 설정
i2c = board.I2C()
oled = adafruit_ssd1306.SSD1306_I2C(128, 64, i2c)

# 화면 초기화
oled.fill(0)
oled.show()

# PIL 이미지 객체 생성 (1비트 흑백)
image = Image.new("1", (oled.width, oled.height))
draw = ImageDraw.Draw(image)

# 폰트 설정
font = ImageFont.truetype("/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf", 10)

# BMP180 센서값 읽기
temp, pressure, altitude = bmpsensor.readBmp180()

x, top = 0, -2

while True:
    # 터미널 출력
    print('Temp      = {0:0.2f} *C'.format(temp))
    print('Pressure = {0:0.2f} Pa'.format(pressure))
    print('Altitude = {0:0.2f} m'.format(altitude))

    # OLED 화면에 텍스트 렌더링
    draw.text((x, top), 'Temp={0:0.1f}C'.format(temp), font=font, fill=255)
    draw.text((x, top+12), 'Press={0:0.0f}Pa'.format(pressure), font=font, fill=255)
    draw.text((x, top+24), 'Alt={0:0.1f}m'.format(altitude), font=font, fill=255)

    oled.image(image)
    oled.show()
    time.sleep(2)
```

```
$ python3 Oled.py
```

PIL의 `Image.new('1', (128, 64))`는 128×64 픽셀의 1비트(흑백) 이미지를 생성합니다.
`ImageDraw.Draw(image)`로 그리기 컨텍스트를 만들고, `draw.text()`로 텍스트를 그린 뒤
`oled.image(image)`로 OLED 드라이버에 전달하면 화면이 갱신됩니다.

🔗 *BMP180과 SSD1306 OLED가 I2C 버스에 함께 연결되어 있어도 각각 다른 주소(0x77, 0x3C)를 사용하므로 충돌 없이 독립적으로 동작합니다. `i2cdetect -y 1`로 두 주소가 모두 보이면 정상입니다.*

참고 문헌

서영진, 사물인터넷을 위한 리눅스 프로그래밍 with 라즈베리파이, 제이펍, 2022 — Chapter 3.

정원석, 사물인터넷을 위한 라즈베리파이3 활용, 광문각, 2017 — Chapter 15.

최주호·김재범·정동진, IoT 사물인터넷과 OpenAI 인공지능을 위한 라즈베리파이5 정석 2판, 앤써북, 2024 — Ch05.